CODE-LAYOUT, READABILITY AND REUSABILITY

OptiCrop: Smart Agricultural Production Optimization Engine

Date	03 July 2026
Team ID	SWTID-2026-7253
Team Size	5
Team Leader	Bommidi Deepika
Team Members	G Uday Kumar, Kavali Anu, Pavan Kumar Sowdepalli, Dasarayyagari Sravan Kumar
Maximum Marks	5 Marks

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability[cite: 16]. Well-organized code with proper naming conventions, comments, and modular design ensures that the **OptiCrop: Smart Agricultural Production Optimization Engine** project is highly maintainable, resilient, and easy to extend in future regional ecosystem developments[cite: 16].

Code Layout Checklist

S.NO	Code Quality Parameter	Description	Followed (Yes/No/ Partial)	Remarks
1	**Consistent Indentation**	Uniform spacing/tabs used throughout the code base.	Yes	PEP 8 standards rigorously followed[cite: 16].
2	**Proper File Structure**	Files and folders are logically and intuitively organized.	Yes	Clear separation of core ML modules, data views, and routing scripts[cite: 16].
3	**Meaningful Variable Names**	Variables reflect their biochemical or operational purpose clearly.	Yes	Descriptive variables used for environmental and crop vectors[cite: 16].
4	**Function / Method Names**	Functions are descriptively named using action verbs.	Yes	Functions named distinctly according to logic tasks[cite: 16].
5	**Code Comments**	Inline and block comments explain complex algorithmic logic.	Yes	Important model initialization and scaling logic documented[cite: 16].
6	**Modular Design**	Code is split into reusable components and independent helper modules.	Yes	Reusable preprocessing, predicting, and metric helper functions implemented[cite: 16].

S.NO	Code Quality Parameter	Description	Followed (Yes/No/ Partial)	Remarks
7	**No Redundant Code**	Duplicate or unused code is strictly audited and removed.	Yes	No redundant loops or duplicate calculations found[cite: 16].
8	**Error Handling**	Exceptions and runtime input errors are handled gracefully.	Yes	Try-except blocks used extensively on form inputs and database calls[cite: 16].

Reusable Components / Modules

S.NO	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	**Agricultural Preprocessing Module** (`preprocess.py`)	Python, Pandas, Scikit-learn	Utilized in `train_model.py` and live `predict.py` web endpoints[cite: 16].	**High**
2	**Optimization Engine Module** (`predict.py`)	Python, Scikit-learn, XGBoost	Called for real-time model inference and crop suitability scoring loops[cite: 16].	**High**
3	**Database Layer Module** (`database.py`)	Python, MySQL	Used for farmer authentication, regional land records, and history persistence[cite: 16].	**High**
4	**Frontend View Templates** (`/templates`)	HTML5, CSS3, Bootstrap 5, JS	Shared layout matrix across login, registration, input wizard, and results displays[cite: 16].	**High**
5	**Machine Learning Checkpoint Store**	Scikit-learn, Joblib	Loaded dynamically by the prediction endpoint to score soil matrix arrays[cite: 16].	**High**

Overall Code Quality Assessment

Aspect Evaluated	Rating (1-5)	Comments
Code Layout & Structure	**5**	Well-organized project structure with clean separation of frontend views, backend routes, database entities, and machine learning components[cite: 16].
Readability	**5**	Meaningful variable names, consistent formatting, and highly readable coding practices matching Python standards[cite: 16].
Reusability	**5**	Decoupled modules for environmental data preprocessing, matching logic, and persistence routines greatly simplify long-term maintenance[cite: 16].

Aspect Evaluated	Rating (1-5)	Comments
Documentation / Comments	**5**	Source scripts contain clear explanatory comments and documentation headers to facilitate seamless developer onboarding[cite: 16].
Overall Score	**5 / 5**	**The project demonstrates excellent code organization, readability, modularity, maintainability, and reusability following Software Engineering best practices[cite: 16].**